

Python II

Marwa Jabberi Slama

Plan



Introduction: Python et l'orienté objets

- ▶ Python est un langage orienté objet
- ▶ Un objet est une donnée constituée de diverses propriétés, et pouvant être manipulée par différentes opérations.
- ▶ La programmation orientée objet est le paradigme qui nous permet de définir nos propres types d'objets, avec leurs propriétés et opérations.
- ▶ Nous travaillerons dans ce cours avec la version 3 de Python (version3).

Objet et caractéristiques

- ▶ Un objet est constitué de 3 caractéristiques :
- ▶ **Un type:** qui identifie le rôle de l'objet (int, str et list sont des exemples de types d'objets)
- ▶ **Des attributs:** qui sont les propriétés de l'objet
- ▶ **Des méthodes:** les opérations qui s'appliquent sur l'objet

Exemple de départ

```
#on instancie une variable number de type int ayant la valeur 5
number=5
#numerator est un attribut de number
number.numerator
#variable value de type list
value=[]
#append est une methode de value
value.append(number)
value
```

➞ [5]

- **Toute valeur en Python est donc un objet**

Application

- ▶ Tout objet est associé à un type
- ▶ Un type définit la sémantique d'un objet
- ▶ On sait par exemple que les objets de type `int` sont des nombres entiers, que l'on peut les additionner, les soustraire, etc.
- ▶ Pour la suite de ce cours, nous utiliserons un type `User` représentant un utilisateur sur un quelconque logiciel. Nous pouvons créer ce nouveau type à l'aide du code suivant :

```
▶ class User:
    pass

### Instanciation d'un objet de type User
Skander = User()
### Définition d'attributs pour cet objet
Skander.id = 1
Skander.name = 'skander'
Skander.password = '12345'
print('Bonjour, je suis {}'.format(Skander.name))
print('Mon id est le {}'.format(Skander.id))
print('Mon mot de passe est {}'.format(Skander.password))
```

```
Bonjour, je suis skander.
Mon id est le 1.
Mon mot de passe est 12345.
```

On dit alors que Skander est une instance de User.

Suite application

- ▶ Nous avons instancié un objet nommé Skander, de type User, auquel nous avons attribué trois attributs.
- ▶ Puis nous avons affiché les valeurs de ces attributs.
- ▶ Notez que l'on peut redéfinir la valeur d'un attribut, et qu'un attribut peut aussi être supprimé à l'aide de l'opérateur **del**

```
[4] Skander.password = 'mot de passe plus sécurisé !'  
Skander.password
```

```
'mot de passe plus sécurisé !'
```



```
del Skander.password  
Skander.password
```

```
-----  
AttributeError                                Traceback (most recent call last)  
<ipython-input-5-4ca34009c8d3> in <module>()  
      1 del Skander.password  
----> 2 Skander.password
```

```
AttributeError: 'User' object has no attribute 'password'
```

SEARCH STACK OVERFLOW

Suite application

- ▶ Les méthodes sont les opérations applicables sur les objets
- ▶ Ce sont en fait des fonctions qui reçoivent notre objet en premier paramètre
- ▶ Nos objets User ne contiennent pas encore de méthode, nous découvrirons comment en ajouter dans la suite du cours
- ▶ Mais nous pouvons imaginer une méthode `verif_pwd` (verif password) pour vérifier qu'un mot de passe entré correspond bien au mot de passe de notre utilisateur

```
[12] def verif_pwd(User, password):  
      return User.password == password
```

```
[13] verif_pwd(Skander, 'hello')
```

False

```
▶ verif_pwd(Skander, '12345')
```

True

Les méthodes recevant l'objet en paramètre, elles peuvent en lire et modifier les attributs. Souvenez-vous par exemple de la méthode `append` des listes, qui permet d'insérer un nouvel élément, elle modifie bien la liste en question

Les classes

- ▶ Une **classe** regroupe des fonctions et des attributs qui définissent un objet. On appelle par ailleurs les fonctions d'une classe des " **méthodes** ".
- ▶ Notre classe Voiture
- ▶ La méthode `__init__()` est appelée lors de la création d'un objet
- ▶ `self.nom` est une manière de stocker une information dans la classe. Dans cet exemple on stocke l'attribut nom dans la classe Voiture



```
class Voiture:  
    def __init__(self):  
        self.nom = "Ferrari"
```

Les Objets

- ▶ Un objet est une instance d'une classe
- ▶ On peut créer autant d'objets que l'on désire avec une **classe**

```
▶ class Voiture:  
    def __init__(self):  
        self.nom = "Ferrari"
```

Les attributs de classe

- ▶ Les attributs de classe permettent de stocker des informations au niveau de la classe
- ▶ Ils sont similaires aux variables
- ▶ On peut rajouter des attributs

```
▶ ma_voiture.modele = "250"  
ma_voiture.modele
```

'250'

```
▶ ma_voiture = Voiture()  
ma_voiture.nom
```

'Ferrari'

Les méthodes

- Les méthodes sont des fonctions définies dans une classe.

Les propriétés

- Les propriétés sont généralement utilisées pour changer les valeurs des attributs
- Il existe une convention de passer par des **getter** et des **setter** pour changer la valeur d'un attribut
- Si nous changeons un attribut cela peut également impacter d'autres attributs
- Les setters permettent de faire cette modification une fois pour toute

```
[22] class Voiture:  
    def __init__(self):  
        self.nom = "Ferrari"  
    def donne_moi_le_modele(self):  
        return "250"
```

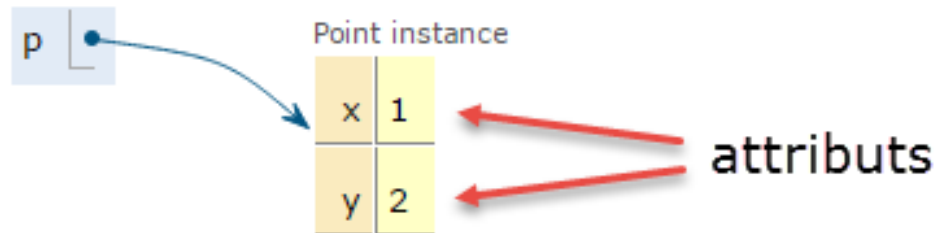
```
▶ ma_voiture=Voiture()  
ma_voiture.donne_moi_le_modele()
```

```
↳ '250'
```

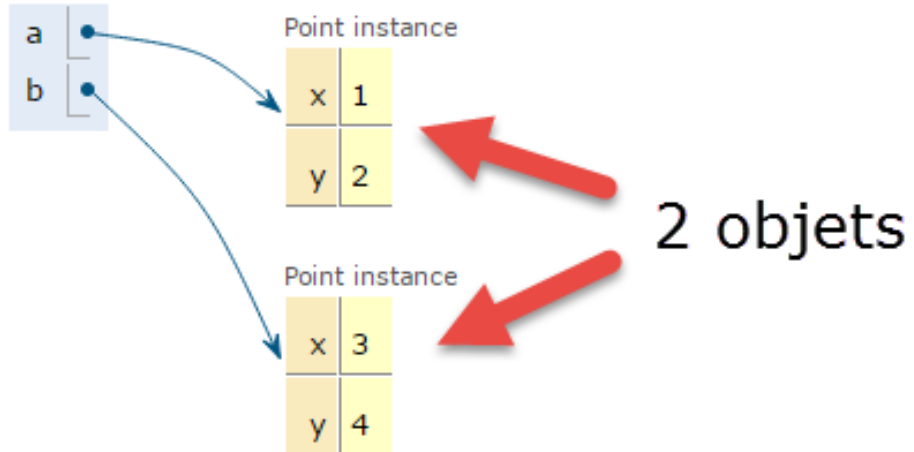
```
▶ class Voiture(object):  
    def __init__(self):  
        self._roues=4  
    def _get_roues(self):  
        print ("Récupération du nombre de roues")  
        return self._roues  
    def _set_roues(self, v):  
        print ("Changement du nombre de roues")  
        self._roues = v  
    roues=property(_get_roues, _set_roues)
```

Exercice

- Créer un objet de type Point ayant deux coordonnées x et y



- Faites la création de deux instances qui représentent deux points a et b



Notion d'encapsulation

- ▶ L'encapsulation est très utile dans la programmation orientée objet
- ▶ Le but est d'éviter une modification par erreur des données d'un objet
- ▶ Il n'y aura pas la possibilité d'agir directement sur les données d'un objet
- ▶ Il est nécessaire de passer par ses méthodes qui jouent le rôle d'interface obligatoire

Attribut privé

- ▶ Pour protéger les attributs de notre classe, les noms des attributs doit débuter par `__`
- ▶ Une fois l'attribut est défini privé ou bien protégé ça ne sera pas possible de lui faire appel depuis l'extérieur de sa classe de base
- ▶ Il faut donc disposer de méthodes qui vont permettre par exemple de modifier ou d'afficher les informations associées à ces variables.

Exemple d'application

```
[29] class Point:
      def __init__(self, x, y):
          self.__x = x
          self.__y = y
```



```
p = Point(1, 2)
p.__x
```

```
-----
AttributeError                                Traceback (most recent call last)
<ipython-input-30-f630f9f53dcc> in <module>()
      1 p = Point(1, 2)
----> 2 p.__x
```

```
AttributeError: 'Point' object has no attribute '__x'
```

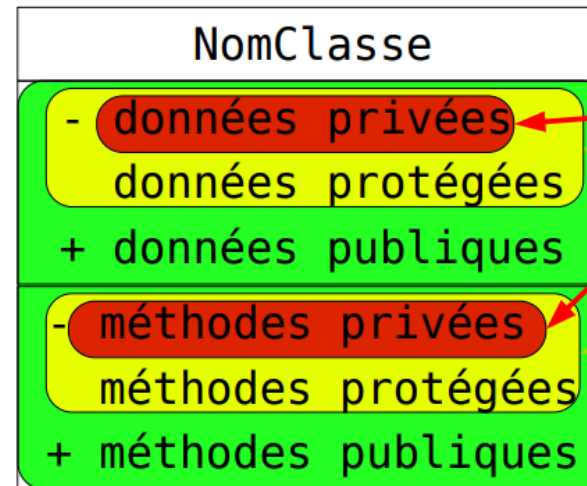
SEARCH STACK OVERFLOW

Suite Exemple

- ▶ Créons des méthodes qui manipulent les attributs protégés ou bien en mode privé
- ▶ Deplace() (pour déplacer les coordonnées)
- ▶ Affiche() (pour afficher les coordonnées)

```
class Point:
    def __init__(self, x, y):
        self.__x = x
        self.__y = y
    def deplace(self, dx, dy):
        self.__x = self.__x + dx
        self.__y = self.__y + dy
    def affiche(self):
        print("abscisse =", self.__x, "ordonnee =", self.__y)
```

```
a = Point(2, 4)
a.affiche()
a.deplace(1, 3)
a.affiche()
```



visibles que au code de la classe

visibles que au code de la classe et des sous-classes

Exercice d'application

- ▶ Faites la création de la classe Point3D ayant x, y et z comme attributs privés
- ▶ Utilisez les getters et les setters pour instancier un Point a

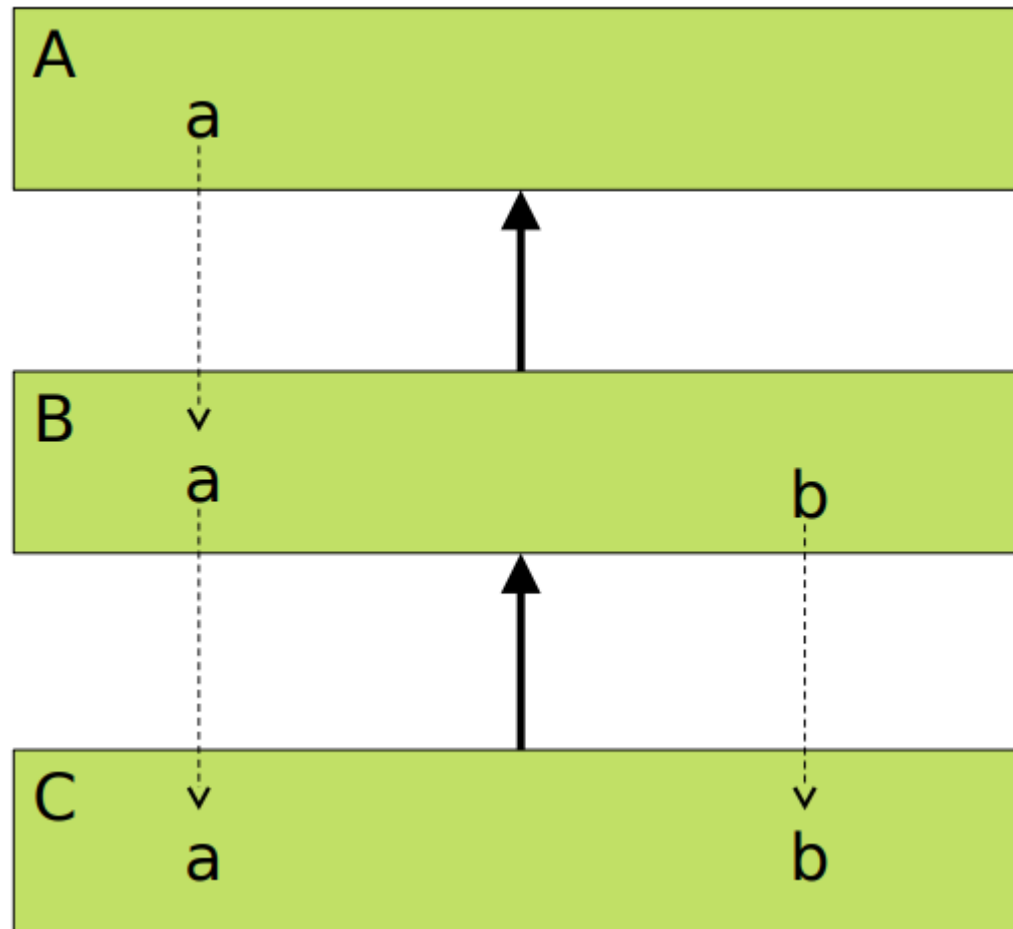
Manipulation des attributs privés

- ▶ Créer une classe Client contenant le nom, le prénom et le mot de passe d'un client
- ▶ Nous souhaitons sécuriser le mot de passe en ajoutant une méthode pour le hasher (à l'aide du module crypt)
- ▶ Ce condensat du mot de passe ne devrait pas être accessible de l'extérieur, et encore moins modifié

NB: Le hashage d'un mot de passe correspond à une opération non-réversible qui permet de calculer un condensat (hash) du mot de passe. Ce condensat peut-être utilisé pour vérifier la validité d'un mot de passe, mais ne permet pas de retrouver le mot de passe d'origine.

L'héritage en python

- L'héritage est la caractéristique d'un langage orienté objet
- Les objets d'une classe "héritent" toutes les propriétés définies pour les classes de niveau supérieur: (attributs et méthodes)



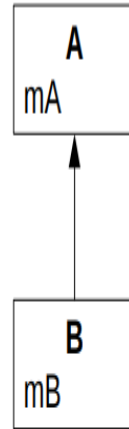
Exemple de démarrage

- Pour dériver la classe B de la class A :

```
class A:  
    def mA(self):  
        print('A')
```

```
class B(A):  
    def mB(self):  
        print('B')
```

```
b = B()  
b.mB()  
b.mA() # Les attributs de A sont accessibles
```



```
class A:  
    def m(self):  
        print('A')
```

```
class B(A):  
    def m(self):  
        print('B')
```

```
b = B()  
b.m() # Affiche "B"
```



La fonction super()

- ▶ C'est une fonction un peu spéciale en Python
- ▶ Elle nous permet d'utiliser la classe parente
- ▶ Elle prend initialement en paramètre une classe et une instance de cette classe
- ▶ Elle retourne un objet qui s'utilise comme une instance de la classe parente

```
class A:  
    def met(self):  
        print('A')  
  
class B(A):  
    def met(self):  
        super().met()  
        print('B')
```

```
x = B()  
x.met()    # Affiche "A" et "B"
```

Exemple d'application

- ▶ Dans cet exemple nous allons étudier l'héritage simple
- ▶ Étendre une unique classe
- ▶ Créer une nouvelle classe (fille) qui bénéficiera des mêmes méthodes et attributs que sa classe mère
- ▶ La définition de nouveaux dans la classe fille n'altèrera pas le fonctionnement de la mère
- ▶ Nous allons étendre notre classe User pour ajouter la possibilité d'avoir des administrateurs
- ▶ Les (Admin) possèdent une nouvelle méthode, manager, pour administrer le système

```
class User:
    def __init__(self, id, name, password):
        self.id = id
        self.name = name
        self._salt = crypt.mksalt()
        self._password = self._crypt_pwd(password)

    def _crypt_pwd(self, password):
        return crypt.crypt(password, self._salt)

    def check_pwd(self, password):
        return self._password == self._crypt_pwd(password)
```

Exemple d'application

```
[2] class Admin(User):  
    def manager(self):  
        print('je suis l administrateur')
```

```
[7] Ska = Admin(1, 'Skander', '1234556')  
Ska.check_pwd('123456')  
Ska.manager()
```

je suis l administrateur

```
▶ Mar = User(2, 'Marwa', '987654')  
Mar.manager()
```

```
-----  
AttributeError                                Traceback (most recent call last)  
<ipython-input-8-ca7578ffaab3> in <module>()  
      1 Mar = User(2, 'Marwa', '987654')  
----> 2 Mar.manager()
```

AttributeError: 'User' object has no attribute 'manager'

SEARCH STACK OVERFLOW

- ▶ Dans la classe mère il existe les méthodes `__init__`, `_crypt_pwd`, et `check_pwd`.
- ▶ Admin a bien hérité les méthodes de la classe mère et en plus il a une méthode `manager` de plus

Qui peut interpréter l'erreur?

Exemple d'application



```
class SimpleUser(User):  
    pass  
class SuperAdmin(Admin):  
    pass
```

- ▶ SimpleUser et Admin sont alors deux classes filles de User
- ▶ L'héritage simple permet aussi d'hériter d'une classe qui hérite elle-même d'une autre classe
- ▶ SuperAdmin est alors la fille de Admin, elle-même la fille de User
- ▶ Donc User est une ancêtre de SuperAdmin

Pour vérifier les parents d'une classe, on peut utilisé l'attribut spécial `__bases__` des classes :



```
Admin.__bases__  
  
SimpleUser.__bases__  
  
SuperAdmin.__bases__
```

Et si on veut chercher `User.__bases__`? Que se passe t il?

Exemple d'application

```
▶ class SimpleUser(User):  
    def __init__(self, id, name):  
        self.id = id  
        self.name = name  
        self._salt = ''  
        self._password = ''  
    def check_pwd(self, password):  
        return True
```

- ▶ L'objectif est étendre les méthodes déjà présentes dans la classe mère
- ▶ La redéfinition est un concept qui permet de remplacer une méthode du parent
- ▶ On veut que la classe SimpleUser ne possède plus aucun mot de passe
- ▶ On modifie alors la méthode check_pwd pour accepter tout mot de passe
- ▶ Il s'agit donc de la redéfinition

- ▶ La méthode __init__ est répété
- ▶ Lorsque nous voudrions apporter des modifications à la méthode de la classe User,
- ▶ On utilise SUPER qui nous permet d'utiliser la classe parente

```
▶ SU = SimpleUser(3, 'simple')  
SU.check_pwd('password')  
####  
super(SimpleUser, SU).check_pwd('password')  
###
```

- ▶ Les arguments de super peuvent être omis ce qui nous permet de simplifier notre méthode __init__ et d'éviter les répétitions

Héritage multiple

- ▶ L'héritage simple permet d'étendre le comportement d'une classe. L'héritage multiple permet de le faire pour plusieurs classes à la fois.
- ▶ Il nous suffit de préciser plusieurs classes entre parenthèses lors de la création de notre classe fille.

```
▶ class Voiture:  
    def assurance(self):  
        return 'Tout risques'  
class Moto:  
    def Pièces(self):  
        return 'Rechange'  
class Transport(Voiture, Moto):  
    pass  
V=Transport()  
V.assurance()  
V.Pièces()
```

- ▶ L'ordre d'héritage est pris en considération

Exercice d'application

- ▶ Nous avons plusieurs classes correspondant à nos fruits favoris, par exemple Argumes, Fruits rouges, Fruits exotiques.
- ▶ Chaque fruit a ses propres propriétés, mais il y a aussi des points communs.
- ▶ Une classe mère Fruit permettant, par exemple, d'instancier un fruit et ajouter des méthodes d'affichage commune à n'importe quel fruit, et ajouter (ou toute autre méthode) pouvant être utilisée pour n'importe quel fruit.
- ▶ Nous pourrions alors créer des classes filles qui héritent de la classe Fruit et ainsi nous économiser des lignes de code identiques à ajouter pour chaque fruit.
- ▶ Les attributs de Fruit: taille, masse, saveur, forme
- ▶ Les méthodes de Fruit: **affiche_conseil** qui affichera tout les détails de chaque fruit instancier.